

Лабораторна робота №3.

Агент з інструментами, пам'яттю і людиною в контурі

2 вересня 2026 р.

Мета

Побудувати агента, який виконує реальні багатокрокові задачі над даними факультету: сам обирає інструменти, тримає стан у графі, переживає падіння процесу і не робить незворотної дії без підтвердження людини.

Робота виконується **бригадою з трьох осіб**.

Модель запускається локально через Ollama, граф — на LangGraph, сховище стану — SQLite.

Склад бригади і розподіл ролей

Кожен учасник відповідає за свою підсистему, має власний результат і захищає його окремо. Спільним є лише інтеграційний крок і підсумковий звіт.

- **Учасник А — інструменти й контракти.** Реалізує набір інструментів із JSON-схемами, типізовані помилки шести класів і таблицю прав із колонкою зворотності.
- **Учасник В — граф, стан і пам'ять.** Переносить цикл у граф станів, підключає checkpointer, реалізує три сховища пам'яті й переривання з підтвердженням людини.
- **Учасник С — прогони й аналіз відмов.** Складає набір задач, ганяє агента, класифікує відмови за таксономією і доводить, які кроки варто винести з моделі в код.

Інтеграційний крок виконують разом: інструменти учасника А підключаються до графа учасника В, після чого набір учасника С проганяється на зібраній системі. Демонстрацію падіння і відновлення показує вся бригада.

Теоретичні відомості

Агент відрізняється від воркфлоу тим, хто вирішує наступний крок: у воркфлоу маршрут намальований людиною, в агенті модель обирає послідовність дій сама. Ціна цієї свободи — необхідність зовнішніх обмежень, бо модель не має власного уявлення про бюджет, права і зупинку.

- **Інструмент є типізованим контрактом:** ім'я, опис для моделі, схема аргументів, обов'язкові поля, перелічувані значення, схема результату. Схема каже *як* викликати, опис каже *коли*. Два майже однакові описи дають нестабільний вибір від запуску до запуску — це перевіряється за три хвилини і запам'ятовується назавжди.

- **Помилка інструмента є вхідними даними для моделі.** Специфікація MCP розділяє протокольні помилки, корисні застосунку, і помилки виконання, які віддають моделі як зворотний зв'язок, придатний до дії. Стек-трейс у контексті забирає крок і не дає нічого. Порожній результат — це успішний виклик, і поводитися з ним треба інакше, ніж зі збоєм.
- **Цикл без критерію зупинки — це нескінченний цикл із чужим бюджетом.** Задаються ліміт кроків, ліміт часу, перелік дозволених дій і детектор повтору. Вичерпання бюджету не мовчить: агент віддає зібране, позначає задачу незавершеною і передає людині.
- **Пам'ять розкладається на три сховища з різним строком життя:** історія діалогу (репліки поточної сесії), робоча пам'ять (план і чернетки до кінця задачі) і довготривала пам'ять (факти, що переживають сеанс). Індекс знань зі спільними документами організації стоїть поруч і пам'яттю не є: у нього інший власник запису. Хибний запис у другому сховищі переживає сеанс і псує всі наступні відповіді — це називають отруєнням пам'яті.
- **Checkpointер зберігає знімок після кожного супер-кроку,** тому відновлення продовжує маршрут, а не починає його. Гарантія при цьому `at-least-once`, а не `exactly-once`: перерваний вузол виконається повторно, а за відкладеного запису (`durability="async"`) можна втратити й останній завершений крок. Захистом від подвоєння є ключ ідемпотентності, обчислений до виклику.
- **Людина в циклі — елемент архітектури, а не кнопка в інтерфейсі.** Гарантією виступає топологія графа: незворотна дія стоїть *після* точки зупинки. Підтвердження, редагування стану і вибір гілки є трьома різними механізмами.

Корисні посилання для ознайомлення:

- Yao S. та ін. [ReAct: Synergizing Reasoning and Acting in Language Models](#), ICLR 2023;
- Anthropic, [Building Effective Agents](#) і [Writing tools for agents](#);
- LangGraph: [Persistence](#), [Human-in-the-loop](#), [Graph API](#);
- Cemri M. та ін. [Why Do Multi-Agent LLM Systems Fail?](#) — таксономія відмов;
- [Claude Code](#) як приклад агента, чию трасу можна прочитати.

Порядок виконання

Позначка [A], [B], [C] вказує відповідального за крок; **[разом]** — крок виконує вся бригада.

1. **[разом] Обрати сценарій** навчального відділу, що вимагає щонайменше трьох інструментів і завершується незворотною дією.
2. **[A] Реалізувати набір інструментів** із явними контрактами і типізованими помилками шести класів. Показати в логах щонайменше одне самовиправлення моделі після такої помилки.
3. **[A] Показати, що опис інструмента впливає на вибір:** два майже однакові описи проти двох розведених, той самий запит, десять прогонів.
4. **[B] Побудувати агента як явний цикл, а потім як граф станів** із типізованим станом. Бюджети кроків і часу мають жити в коді, а не в промпті.

5. **[В] Додати збереження стану і три сховища пам'яті** з явним правилом, що куди пишеться. Показати, що агент переживає падіння процесу і що підтверджений запис не задвоюється.
6. **[В] Поставити людину в контур** перед незворотною дією так, щоб заборона трималася топологією графа, а не проханням у промпті.
7. **[С] Прогнати набір із 25 задач** у кількох повторах, з яких частина потребує кількох інструментів, а частина свідомо нерозв'язна. Кожну відмову віднести до класу таксономії відмов із переліку літератури.
8. **[разом] Оформити ноутбук**. Зібрати роботу в один Jupyter-ноутбук: код кожного учасника у своїх комірках із короткими коментарями. Обов'язково всередині: оголошення інструментів, схема графа, лог демонстрації падіння і відновлення з поясненням, які вузли виконались повторно, таблиця прогонів, таблиця відмов за класами і висновок до 250 слів про те, які кроки варто винести з моделі в детермінований код.

Контрольні запитання

1. За якою ознакою задачу віддають агенту, а не workflow?
2. Чому опис інструмента вважають частиною промпта?
3. Що саме повертає модель, коли «викликає інструмент»?
4. Чому порожній результат виділяють окремо від збою сервісу?
5. Що має статися при вичерпанні бюджету кроків і чого статися не має?
6. Куди записати, що студент навчається в групі КН-21, і чому не в історію діалогу?
7. Після відновлення з checkpoint запис створився двічі. Де помилка?
8. Чим `durability="sync"` відрізняється від `"async"` з погляду демонстрації?
9. Чому підтвердження людини реалізують у графі, а не інструкцією в промпті?
10. Агент повторює той самий пошук п'ять разів поспіль. Які три причини найімовірніші?
11. Чому побічний ефект має стояти після точки зупинки, а не перед нею?

Література

1. Shinn N. та ін. [Reflexion: Language Agents with Verbal Reinforcement Learning](#). NeurIPS, 2023.
2. Packer C. та ін. [MemGPT: Towards LLMs as Operating Systems](#). arXiv:2310.08560, 2023.
3. Model Context Protocol. [Server: Tools](#).
4. OWASP GenAI Security Project. [LLM06: Excessive Agency](#), 2025.